

Optimizing SPFA via Tarjan's Subtree Disassembly Trick

Yanqiao Chen

Southern University of Science and Technology
Department of Computer Science and Engineering
12412115@mail.sustech.edu.cn

May 11, 2026

1 Tarjan's Subtree Disassembly Trick

1.1 Observation: Why it works, intuitively?

Tarjan's Subtree Disassembly Trick[1] is a useful technique for optimizing the Shortest Path Faster Algorithm (SPFA) by decomposing the graph into subtrees. Each node in the graph preserves a pointer to its parent node, and thus it forms a tree structure that represents the shortest paths.

Each time when we need to relax an edge (u, v) , we immediately find u in the subtree of v instead of backtracking. Once we found u , it means that there exists a negative cycle.

If there is no negative cycle, we delete the whole subtree of v and annotate all the nodes as not discovered and pop them out of the queue. This is because now the estimation base on the new $d(v)$, thus all the estimation made by the previous $d(v)$ is invalid.

The algorithm made very clever pruning method: First, the algorithm will look ahead whether the node in the queue is useful or not. If a v is updated frequently, then nodes will be in-queued frequently, leading to a efficiency degradation.

1.2 Pseudocode

For comparison, we first list the standard SPFA in Algorithm 1. The Tarjan-optimized version is given in Algorithm 4.

We provide the auxiliary function in Algorithm 2 and Algorithm 3 to preserve the subtree structure.

Algorithm 1 Standard SPFA

Require: Graph $G = (V, E)$, source node s

Ensure: Distance array d , or report a negative cycle

```
1: Initialize  $d[v] \leftarrow +\infty$ ,  $\text{count}[v] \leftarrow 0$ ,  $\text{inqueue}[v] \leftarrow \text{false}$  for all  $v \in V$ 
2:  $d[s] \leftarrow 0$ 
3:  $Q \leftarrow \text{Queue}(\{s\})$ ,  $\text{inqueue}[s] \leftarrow \text{true}$ 
4: while  $Q$  is not empty do
5:    $u \leftarrow Q.\text{dequeue}()$ 
6:    $\text{inqueue}[u] \leftarrow \text{false}$ 
7:   for each edge  $(u, v)$  with weight  $w$  do
8:     if  $d[u] + w < d[v]$  then
9:        $d[v] \leftarrow d[u] + w$ 
10:      if  $\neg \text{inqueue}[v]$  then
11:         $Q.\text{enqueue}(v)$ 
12:         $\text{inqueue}[v] \leftarrow \text{true}$ 
13:         $\text{count}[v] \leftarrow \text{count}[v] + 1$ 
14:        if  $\text{count}[v] > |V|$  then
15:          return "Negative cycle detected"
16:        end if
17:      end if
18:    end if
19:  end for
20: end while
21: return  $d$ 
```

Algorithm 2 Auxiliary Procedure: IS-IN-SUBTREE(u, v)

```
1: function IS-IN-SUBTREE( $u, v$ )
2:    $\text{curr} \leftarrow v$ 
3:   while  $\text{curr} \neq \text{null}$  do
4:     if  $\text{curr} = u$  then
5:       return true
6:     end if
7:      $\text{curr} \leftarrow \text{parent}[\text{curr}]$ 
8:   end while
9:   return false
10: end function
```

Algorithm 3 Auxiliary Procedure: DELETE-SUBTREE(x)

```
1: function DELETE-SUBTREE( $x$ )
2:    $S \leftarrow \text{Stack}(\text{children}[x])$ 
3:    $\text{children}[x] \leftarrow \emptyset$ 
4:   while  $S$  is not empty do
5:      $y \leftarrow S.\text{pop}()$ 
6:      $d[y] \leftarrow +\infty$ 
7:      $\text{inqueue}[y] \leftarrow \text{false}$ 
8:      $\text{parent}[y] \leftarrow \text{null}$ 
9:     for each  $z \in \text{children}[y]$  do
10:       $S.\text{push}(z)$ 
11:     end for
12:      $\text{children}[y] \leftarrow \emptyset$ 
13:   end while
14: end function
```

Algorithm 4 SPFA with Tarjan's Subtree Disassembly Trick

Require: Graph $G = (V, E)$, source node s

Ensure: Distance array d , or report a negative cycle

```
1: Initialize  $d[v] \leftarrow +\infty$ ,  $\text{parent}[v] \leftarrow \text{null}$ ,  $\text{children}[v] \leftarrow \emptyset$ ,  $\text{inqueue}[v] \leftarrow \text{false}$  for all  $v \in V$ 
2:  $d[s] \leftarrow 0$ 
3:  $Q \leftarrow \text{Queue}(\{s\})$ ,  $\text{inqueue}[s] \leftarrow \text{true}$ 
4: while  $Q$  is not empty do
5:    $u \leftarrow Q.\text{dequeue}()$ 
6:    $\text{inqueue}[u] \leftarrow \text{false}$ 
7:   for each edge  $(u, v)$  with weight  $w$  do
8:     if  $d[u] + w < d[v]$  then
9:       if IS-IN-SUBTREE( $u, v$ ) then
10:        return "Negative cycle detected"
11:      end if
12:      DELETE-SUBTREE( $v$ )
13:      if  $\text{parent}[v] \neq \text{null}$  then
14:         $\text{children}[\text{parent}[v]].\text{remove}(v)$ 
15:      end if
16:       $\text{parent}[v] \leftarrow u$ 
17:       $\text{children}[u].\text{add}(v)$ 
18:       $d[v] \leftarrow d[u] + w$ 
19:      if  $\neg \text{inqueue}[v]$  then
20:         $Q.\text{enqueue}(v)$ 
21:         $\text{inqueue}[v] \leftarrow \text{true}$ 
22:      end if
23:    end if
24:  end for
25: end while
26: return  $d$ 
```

1.3 What makes it different: The key idea of Tarjan's trick

Let us consider a bottleneck in standard SPFA. Suppose node u is dequeued and relaxes an edge to its neighbor v , yielding a smaller $d[v]$. Node v is then enqueued. However, if v already had a subtree of descendants in the current shortest-path tree, those descendants retain distance estimates derived from the obsolete value of $d[v]$. They may still reside in the queue (or be processed later through other updates) and continue propagating those stale values, causing redundant relaxations. When v has a large subtree, this waste becomes severe.

Tarjan's Subtree Disassembly Trick eliminates this inefficiency immediately. As soon as v 's distance is updated, the algorithm invokes `DELETE-SUBTREE( $v$ )`: it traverses v 's entire old subtree, resets every descendant's distance to $+\infty$, clears their parent pointers, removes them from the queue if present, and discards their child lists. Only after this aggressive pruning is v linked to its new parent u and enqueued. By tearing down the obsolete subtree instantly, the trick ensures that no node ever propagates a stale distance estimate, which drastically reduces the number of useless relaxations.

This idea can be seen as a form of eager pruning. In the shortest-path tree maintained by the algorithm, every descendant's distance is inductively derived from its ancestors: each node's estimate equals the total edge weight along the unique tree path from the source. Thus the validity of the entire subtree is contingent on the root's distance remaining unchanged. The instant $d[v]$ is updated, all prior estimates in v 's subtree become obsolete. Instead of letting these stale values linger in the queue and trigger useless relaxations, the algorithm discards the subtree immediately. This ensures that every pending relaxation is grounded in a fresh, valid distance estimate.

2 The Complexity Analysis

2.1 Amortized Analysis

We analyze the amortized cost of a successful relaxation. In `DELETE-SUBTREE( $v$ )`, every node in the old subtree rooted at v is visited a constant number of times (popped from the stack, reset, and its children are pushed). Thus the actual cost of deleting a subtree of size s is at most $c \cdot s$ for some fixed constant $c \geq 1$.

We use the potential method with

$$\Phi = c \cdot (\text{total number of nodes currently in the shortest-path tree}).$$

Equivalently, $\Phi = c \cdot \sum_{v \in V} |\text{children}[v]|$.

Consider a successful relaxation of edge (u, v) . Let s be the size of the old subtree rooted at v .

- **Delete-Subtree:** The actual cost is at most $c \cdot s$. The subtree is removed from the tree, so the potential drops by $c \cdot s$.
- **Re-attach v :** The actual cost is $O(1)$. Node v is added as a child of u , so the potential increases by c .

The net change in potential is $\Delta\Phi = -c \cdot s + c$. Therefore the amortized cost is

$$\hat{c} = c_{\text{actual}} + \Delta\Phi \leq (c \cdot s + O(1)) + (-c \cdot s + c) = O(1).$$

Thus every successful relaxation has $O(1)$ amortized cost. Since each node's distance can be updated at most $O(|V|)$ times in a graph with no negative cycles, the number of successful

relaxations is $O(|V| \cdot |E|)$, and the total amortized cost of all successful relaxations is $O(|V| \cdot |E|)$. The practical gain of Tarjan’s trick comes from eliminating useless relaxations by stale nodes, not from improving the asymptotic worst-case bound.

The worst case follows the amortized complexity, which is also $O(|V| \cdot |E|)$.

2.2 Space Complexity

The complexity of this algorithm is simple. We only have to preserve a double direction link list ($O(n)$) for the tree, the adjacent list ($O(m)$) and other auxiliary arrays (distance, and a queue, $O(n)$). Thus the total space complexity is $O(m + n)$.

3 In what case do the new algorithm performs better

Let us construct a simple case such that the SPFA degrades.

Graph construction. Consider the graph G illustrated in Figure 1. It contains a source s , an intermediate node a , a hub node h , and n leaf nodes $\ell_1, \dots, \ell_n$. The edge weights are chosen so that h is first reached via the direct edge $s \rightarrow h$ with distance 0, and later via the detour $s \rightarrow a \rightarrow h$ with a strictly smaller distance -5 . Every leaf ℓ_i is attached to h with weight 0.

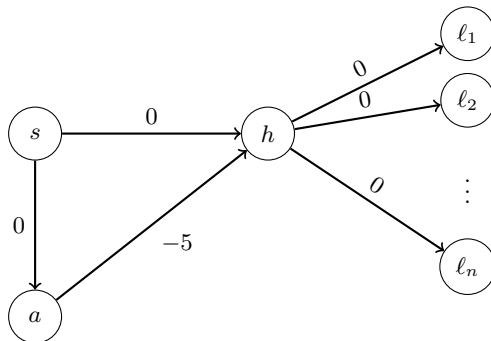


Figure 1: A degenerate case for standard SPFA. For simplicity, the node h represents a node that will be frequently updated, while the number of the downstream nodes is large. We will not draw the complete graph for simplicity.

This star-shaped structure is deliberately simple; the key is to imagine that each leaf ℓ_i is the entry point to a downstream subgraph (a chain, a tree, or a dense component). The downstream structure is omitted from the figure for clarity, but it is where the inefficiency manifests.

When h is relaxed, all its descendants are enqueued for subsequent processing. If h is relaxed again later, a new batch of descendants is enqueued with improved distances, while the previous copies—still residing in the queue—continue to carry stale estimates. These obsolete nodes then propagate incorrect information through their downstream subgraphs, causing cascading redundant work. Tarjan’s Subtree Disassembly Trick avoids this entirely by deleting the entire obsolete subtree the moment h is updated, pruning all stale descendants before they can perform any useless relaxations.

This example demonstrates that the gain of Tarjan’s Subtree Disassembly Trick is not a better asymptotic bound, but the elimination of massive redundant work by eagerly invalidating stale subtrees before they can cascade.

Remark 1. When the graph contains a negative cycle, Tarjan’s algorithm detects it via `IS-IN-SUBTREE( $u, v$ )`, which traverses only the ancestors of v in the current shortest-path tree. Standard SPFA, by contrast, relies on counting enqueue operations (`count[ $v$ ] > |V|`) and may require up to $O(|V|)$ updates per node before detection. In practice, the Tarjan version can discover negative cycles earlier because the check is performed immediately upon relaxation rather than after a threshold is reached. This is a modest but tangible efficiency improvement. However, both algorithm can be hack by specific graph, thus the worst case complexity won’t change.

4 Conclusion

Tarjan’s Subtree Disassembly Trick is a pragmatic heuristic for SPFA. It does not improve the asymptotic worst-case or amortized complexity; the theoretical bound remains $O(|V| \cdot |E|)$. The practical gain comes from eliminating cascading useless relaxations, not from a smaller constant factor. There is an inherent trade-off: every successful relaxation pays an additional $O(s)$ cost for `DELETE-SUBTREE` (where s is the size of the obsolete subtree), but in return it avoids $\Theta(s \cdot \bar{w})$ wasted work that stale nodes would otherwise inflict on their downstream subgraphs. This trade-off is favorable on graphs where nodes are updated frequently or possess large subtrees, which explains why the Tarjan-optimized variant often outperforms standard SPFA in practice.

References

- [1] Ahmed Shokry. Shortest path algorithms between theory and practice, 2019.